

TensorFlowLite-MovEdge

Clasificador de movimientos

Descripción de la Actividad.

En este microproyecto continuaremos trabajando con el **clasificador de movimientos**, sistema que identifica y categoriza distintos tipos de movimientos o gestos realizados por un objeto o persona basándose en datos de entrada, usualmente provenientes de sensores. Recordando las principales etapas tenemos:

Funcionamiento general:

- * **Recopilación de Datos:** El clasificador comienza con la recopilación de datos de movimiento, generalmente a través de sensores como acelerómetros, giroscopios, o cámaras. Estos dispositivos capturan la dinámica del movimiento en forma de datos crudos que pueden incluir aceleración, orientación, posición, entre otros.
- * **Preprocesamiento:** Los datos recogidos se procesan para eliminar el ruido y extraer características relevantes que describen los movimientos de manera efectiva. Esto puede incluir la normalización, el filtrado, o la transformación de los datos.
- * **Extracción de Características:** Se extraen características significativas de los datos preprocesados, como la amplitud máxima, la frecuencia dominante, o los valores medios. Estas características ayudan a distinguir entre diferentes tipos de movimientos.
- * **Entrenamiento del Modelo:** Se utiliza un conjunto de datos etiquetado para entrenar un modelo de aprendizaje automático. Este modelo aprende a asociar patrones de características con categorías específicas de movimiento.
- * **Clasificación:** Una vez entrenado, el modelo puede clasificar nuevos datos de movimiento en tiempo real o en lotes, asignando cada segmento de datos a una categoría de movimiento basada en sus características.

En esta actividad nos enfocaremos en la parte de entrenamiento del modelo basando la implementación en el Framework de TensorFlow, para luego usar TensorFlow Lite. En este tercer caso de estudio realizaremos todas las etapas del proceso basados en los ejercicios realizados previamente implementando la solución en la tarjeta de desarrollo ESP32.

Resultados de aprendizaje

Esta actividad ofrece una oportunidad para explorar el uso de tarjetas de desarrollo como la ESP32 y el uso de las librerías de TensorFlow Lite. Los estudiantes aprenderán a dar solución a un sistema de clasificación usando estas librerías. Al final de la actividad, los estudiantes deberían ser capaces de:

- * Implementar un sistema de reconocimiento de movimientos y optimizar su solución para implementarla en una ESP32 por medio de las librerías de TensorFlow Lite.
- * Identificar los diferentes algoritmos de optimización usados por TensorFlow Lite y comparar los resultados de inferencia respecto a modelos sin optimización.

Materiales Necesarios.

HARDWARE:

- * 1 ESP32 DEVKIT V1. (Tarjeta o placa de programación)
- * 1 MPU6050. (Sensor acelerómetro).
- * 1 OLED 0.96 inch SSD1306 128x32 4 pin I2C (Display) OPCIONAL PARA ESTA ACTIVIDAD.
- * 1 juego cables (jumpers).
- * 1 Protoboard.

SOFTWARE:

- * Visual Studio Code con plugin PlatformIO.
- * Python.

Marco Teórico.

TensorFlow Lite es una herramienta optimizada de TensorFlow diseñada para la ejecución de modelos de machine learning en dispositivos móviles y sistemas embebidos. Facilita la implementación de modelos en entornos con recursos limitados, permitiendo inferencias eficientes y de baja latencia. Consta de dos componentes principales: el intérprete y el convertidor. El convertidor transforma los modelos entrenados de TensorFlow en un formato más compacto y optimizado (.tflite), mientras que el intérprete ejecuta estos modelos en dispositivos de borde, utilizando técnicas como la cuantización para mejorar la eficiencia. Una de las características clave de TensorFlow Lite es la capacidad de optimizar modelos mediante técnicas como la cuantización, que reduce la precisión de los pesos y las activaciones del modelo, disminuyendo así el tamaño del modelo y el consumo de energía sin sacrificar significativamente la precisión.

TensorFlow Lite es ampliamente utilizado en aplicaciones de visión por computadora, reconocimiento de voz, procesamiento de texto y otros dominios donde se requiere la inferencia en tiempo real en dispositivos como teléfonos móviles, dispositivos IoT y wearables. Al utilizar TensorFlow Lite, es importante considerar las limitaciones de hardware del dispositivo de destino, como la memoria y la potencia de procesamiento. Además, se debe evaluar la compensación entre la precisión del modelo y la eficiencia al aplicar técnicas de optimización como la cuantización.

Metodología.

El desarrollo de esta actividad es individual por lo que cada estudiante debe tener todos los materiales necesarios para su desarrollo. Basados en los resultados del microproyecto anterior y en las técnicas de preprocesamiento estudiadas, se busca implementar un sistema de reconocimiento de movimientos. Es posible construir una solución usando las librerías de TensorFlow, para luego implementar la solución en usando TensorFlow lite en el microcontrolador ESP32.

Captura de datos:

- * Conecte la ESP32 con el sensor MPU6050 y la pantalla OLED como se hizo en las entregas anteriores. Verifique que se obtienen los datos del acelerómetro y giróscopo en la consola serial de su computador.
- * Ejecute el código de la ESP32 y el script de Python `data_acquisition.py` en su computador para capturar datos del acelerómetro y giroscopio para diferentes movimientos. Este script permite guardar secuencialmente los valores del sensor diferenciando 4 movimientos y una condición de no movimiento, para crear una pequeña base de datos etiquetada. Realice una captura de datos de los movimientos los cuales puede ser: Mov00 (quieto), Mov01 (hacia arriba), Mov02 (hacia abajo), Mov03 (hacia la izquierda), Mov04 (hacia la derecha) o movimientos propuestos por usted pero manteniendo el Mov00 (quieto) propuesto. Este Mov00 (quieto) o reposo se sugiere por su facilidad de reconocimiento y porque permite organizar el protocolo para validar el sistema. Los datos quedarán guardados en un archivo csv con el nombre `data.csv`.

Clasificación de movimientos:

- * Implemente un clasificador utilizando un modelo con las librerías de TensorFlow usando python. Recuerde utilizar preprocesamiento de datos según las técnicas estudiadas previamente para lograr mejores resultados en su inferencia.
- * Realice pruebas controladas con los archivos csv de datos.

Implementación clasificador en la ESP32 en tiempo real:

- * Implemente el clasificador de movimientos en la ESP32 usando TensorFlow Lite. **El clasificador debe quedar implementado en la ESP32, para reconocer los movimientos propuestos en tiempo real.**
- Realice pruebas de funcionamiento del clasificador de movimientos en la ESP32. Genere un protocolo de pruebas para validar el funcionamiento de su clasificador de movimientos. Protocolo se refiere a pruebas controladas, donde el sistema sepa el movimiento que se está haciendo y el reconocedor pueda identificarlo.
- Genere un archivo raw con la información de los datos del acelerómetro y giróscopo de la ESP32, un identificador para cada movimiento ejecutado, uno para el movimiento reconocido.

Entregables.

- * Se propone realizar el siguiente protocolo de pruebas: se debe generar una rutina controlada por tiempo, donde se realicen repeticiones de cada movimiento durante 15 segundos en el siguiente orden: Mov00 (quieto), Mov01, Mov00 (quieto), Mov02, Mov00 (quieto), Mov03, Mov00 (quieto), Mov04, Mov00 (quieto). Cada movimiento se debe hacer durante 15 segundos. Luego se debe generar una secuencia aleatoria de movimientos incluido Mov00 (quieto), por periodos de 5 segundos. Esto durante 30 segundos.
- * Al final de la rutina se debe generar un archivo de texto con los resultados obtenidos data.csv. El archivo raw debe tener los datos del acelerómetro y giróscopo; el movimiento ejecutado, el movimiento reconocido y el resultado de la inferencia calculada para cada movimiento.
- * Entregue un informe en formato libre donde explique toda su estrategia de implementación detallando el algoritmo de preprocesamiento utilizado y la definición del modelo de reconocimiento implementado.
- * Entregue los archivos main.cpp (archivo del ESP32) y data.csv con el formato de la tabla presentada a continuación.

Feature1	Feature2	Feature3	Feature4	Feature5	Feature6	Feature7	Clase Real	Clase Predecida	Resultado Inferencia
-2.3	8.78	6.08	-1.04	-0.73	0.06	22.57	Mov04	Mov04	0.000148
-3.8	8.55	3.1	0.36	0.01	-0.03	23.54	Mov01	Mov02	0.034273

donde:

Feature1 = Accel x (m/s²)

Feature2 = Accel y (m/s²)

Feature3 = Accel z (m/s²)

Feature4 = w x (rad/s)

Feature5 = w y (rad/s)

Feature6 = w z (rad/s)

Feature7 = Temp (oC)

Clase Real = identificador del movimiento ejecutado

Clase Predecida = identificador del movimiento reconocido

Resultado Inferencia = Resultado de la inferencia (% reconocimiento)

Es importante poder llegar a una solución que permita diferenciar los movimientos de manera clara y precisa.

Evaluación.

Antes de la entrega deben diligenciar esta lista de chequeo para asegurar la calidad de su trabajo.

1. Preprocesamiento:

- ☐ Se presentan la selección de algoritmos utilizados para el preprocesamiento de forma clara y detallada.
- ☐ Se validan los resultados obtenidos usando dichos algoritmos de preprocesamiento de forma que es clara su selección y uso.
- ☐ Se presentan claramente métricas de comparación para cada caso.

2. TensorFlow LITE:

- ☐ Se presenta de forma general el algoritmo basado en TensorFlow Lite, explicando la estrategia de solución.
- ☐ Se proponen y presentan métricas y protocolo adecuados para la validación de los resultados en continuo.

3. Archivos de trabajo:

- ☐ Se entregan los archivos main.cpp y data.csv con el formato solicitado.